



Universidad Autónoma de Chihuahua

Facultad de Ingeniería

Bases de Datos Avanzadas

Maestro a cargo. - Ing. De Lira Mmiramontes Jose Saul

Proyecto 2 parcial base de Autos en MongoDB

Nombre del alumno -.

Alisandro Mendoza Espitia - 364693

Marcos Iram Casas Mora - 361853

Alexis Aidan Leon Rodelas - 377171

Grupo. - 7CC2

Fecha de elaboración. - 30 de octubre del 2026

Lugar de elaboración. - Chihuahua, Chihuahua.

Introducción

El análisis de datos moderno exige arquitecturas de almacenamiento capaces de procesar grandes volúmenes de información con esquemas flexibles y altos niveles de escalabilidad. Tradicionalmente, las organizaciones han operado bajo el modelo relacional (SQL), el cual garantiza consistencia ACID estricta a costa de una alta rigidez estructural y la necesidad de ejecutar operaciones de unión (*JOINs*) computacionalmente costosas a medida que el volumen de datos incrementa.

El propósito de este proyecto es realizar la migración completa de la base de datos relacional BDAutos hacia una arquitectura NoSQL orientada a documentos utilizando MongoDB y el controlador oficial PyMongo en Python. La solución no solo contempla el rediseño del modelo de datos para optimizar el rendimiento analítico mediante el *Aggregation Framework*, sino que también implementa una aplicación con interfaz gráfica (Tkinter) y un módulo de administración por terminal orientado al mantenimiento seguro de la información mediante borrado lógico.

Objetivos del Proyecto

- Diseño e Integración NoSQL: Reestructurar un modelo relacional de 6 tablas hacia una arquitectura de 4 colecciones de documentos optimizadas para consulta.
- Migración ETL: Automatizar el proceso de Extracción, Transformación y Carga (ETL) desde datos fuente hacia MongoDB mediante scripts en Python (init_db.py).
- Analítica Avanzada: Desarrollar 25 canalizaciones BSON (*pipelines*) de agregación para responder a requerimientos de negocio en tiempo real.
- Desarrollo de Software Modular: Implementar una arquitectura desacoplada en Python dividida en interfaz gráfica (gui.py), lógica de acceso a datos (mongo_bridge.py), catálogo BSON (queries_catalog.py) y gestión por terminal (crud_manager.py).

Modelado de Datos: Mapeo Relacional a Documentos

En el diseño NoSQL existen dos estrategias fundamentales para estructurar relaciones: Embebido (*Embedded/Denormalization*) y Referenciado (*Linking/Normalized Links*). La elección entre ambas depende del patrón de acceso a los datos, la cardinalidad de las entidades y el tamaño final de los documentos.

Criterios de Selección: Embebido vs. Referenciado

- Documentos Embebidos (*Embedded*): Se utiliza cuando los datos relacionados se leen de forma conjunta en la mayoría de las operaciones analíticas y la relación es de 1:1 o 1:N acotada. En BDAutos, la tabla relacional cars_data (especificaciones técnicas como MPG, potencia y peso) y la tabla car_names (descripción del vehículo) se fusionaron en un solo documento dentro de la colección cars. Esto elimina la necesidad de realizar uniones en tiempo de ejecución, permitiendo recuperar el perfil completo de un auto en una sola lectura de disco.
- Referencias de Documentos (*Linking*): Se utiliza cuando la entidad referenciada posee una cardinalidad alta, es consultada de forma independiente o es compartida por múltiples entidades superiores. En BDAutos, se conservaron referencias explícitamente mediante claves externas (continentID, countryID, modelID) en las colecciones countries, makers y cars. Esta estrategia evita la duplicación masiva de texto (como nombres de países o fabricantes) y mantiene la integridad cuando se requieren reportes jerárquicos globales mediante la etapa $lookup.

Transformación de Esquema SQL a Colecciones MongoDB

Tabla SQL Original	Colección MongoDB	Estrategia Aplicada	Justificación Técnica
`continents`	`continents`	INDEPENDIENTE	Entidad raíz con cardinalidad baja (5 continentes). Sirve como catálogo de consulta general.
`countries`	`countries`	REFERENCIADO (LINKING)	Conserva `continentID` para conectar con `continents`. Permite filtrados por región sin duplicar datos.
`car_makers`	`makers`	REFERENCIADO (LINKING)	Conserva `countryID` para vincular el fabricante con su país de origen.
`model_list`	`makers` / `cars`	REFERENCIA INTERMEDIA	Maneja el identificador `modelID` que conecta la marca comercial con la especificación del auto.
`car_names`	`cars`	EMBEBIDO TOTAL	El nombre/descripción del vehículo se embebió directamente como el campo `descr` en la colección `cars`.
`cars_data`	`cars`	EMBEBIDO TOTAL	Los atributos numéricos (MPG, cilindros, potencia, peso, aceleración) forman el núcleo del documento `cars`.

Mapeo de Campos de la Colección Principal (cars)

Campo MongoDB	Tipo de Dato	Origen Relacional	Atributo / Función Técnica
`_id`	`Integer`	`car_names.MakeId`	Clave primaria única del vehículo.
`modelID`	`Integer`	`model_list.ModelId`	Clave foránea de referencia hacia la marca/ modelo.
`descr`	`String`	`car_names.Descr`	Descripción o nombre comercial del auto.
`year`	`Integer`	`cars_data.Year`	Año de fabricación del modelo.
`mpg`	`Double`	`cars_data.MPG`	Rendimiento de combustible en millas por galón.
`cylinders`	`Integer`	`cars_data.Cylinders`	Cantidad de cilindros del motor.
`horsepower`	`Double`	`cars_data.Horsepower`	Potencia del motor en caballos de fuerza.
`weight`	`Double`	`cars_data.Weight`	Peso total del vehículo en libras.
`accel`	`Double`	`cars_data.Accelerate`	Tiempo de aceleración de 0 a 60 mph en segundos.
`"activo"`	`Boolean`	*Atributo NoSQL Nuevo*	Flag de estado para el control de **borrado lógico** (`True` / `False`).

Configuración del Entorno Virtual (venv)

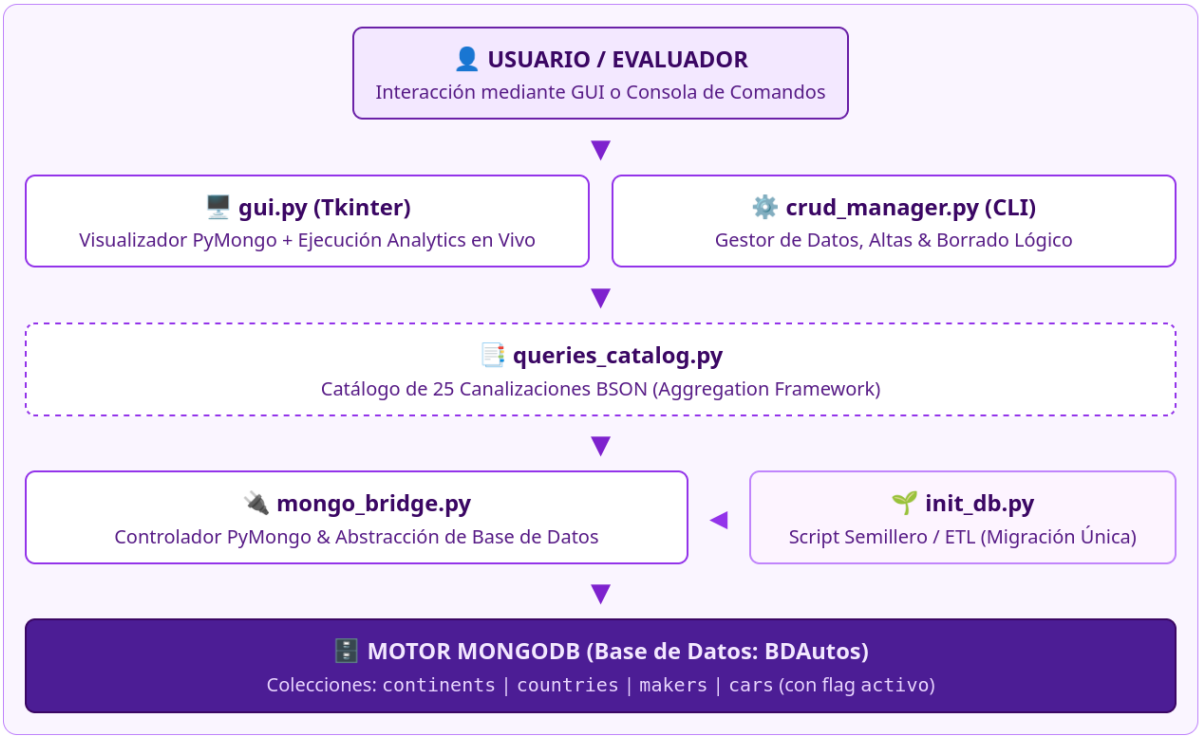
En distribuciones modernas de Linux (como Ubuntu, Debian o Kubuntu), la especificación PEP 668 marca el entorno de Python del sistema operativo como administrado externamente (*externally-managed environment*). Esto impide la instalación directa de paquetes vía pip global para evitar la corrupción de dependencias críticas del sistema base.

Para garantizar la portabilidad y aislamiento del proyecto, se configuró un entorno virtual (venv):

- Aislamiento de Dependencias: El proyecto ejecuta pymongo de forma aislada sin alterar las librerías del sistema operativo.
- Reproducibilidad: Cualquier evaluador o servidor de despliegue puede replicar exactamente el mismo ambiente de ejecución en Windows, macOS o Linux mediante el comando python -m venv venv.
- Seguridad: Previene conflictos entre versiones de PyMongo u otros controladores de base de datos.

Arquitectura Modular "Code as Object"

El código fuente se diseñó bajo el principio de Separación de Responsabilidades (*Separation of Concerns*) y la metodología orientada a objetos ("Code as Object"), distribuyendo el sistema en 5 componentes independientes:



- init_db.py: Se encarga de la extracción de datos relacionales planos, su estructuración hacia colecciones NoSQL BSON y el poblamiento inicial de MongoDB. Se ejecuta una sola vez al inicio.

- mongo_bridge.py (Capa de Acceso a Datos): Encapsula la conexión con el servidor MongoDB (localhost:27017), proveyendo métodos estándar para la ejecución de agregaciones, búsquedas y operaciones de actualización.

- queries_catalog.py (Catálogo de Consultas BSON): Modula los 25 requerimientos analíticos en objetos estructurados (QueryRequirement). Cada objeto almacena el título, descripción, código PyMongo en texto plano para visualización en la GUI y la función ejecutable BSON.

- gui.py (Capa de Presentación / Analytics): Construida en Tkinter, permite seleccionar requerimientos, inspeccionar el código de la consulta en tiempo real, ejecutar pruebas ad-hoc estilo consola de MongoDB Compass y desplegar los resultados en tablas dinámicas.

- crud_manager.py (Gestor de Datos por Terminal): Programa independiente de consola diseñado para administrar el ciclo de vida de los registros (creación, edición, borrado lógico y restauración) sin recargar la interfaz gráfica analítica.

Principios de Conservación de Datos

En entornos empresariales de producción, la eliminación física de registros (delete_one / delete_many) se considera una práctica de alto riesgo por las siguientes razones:

- Integridad Histórica y Auditoría: Borrar un documento destruye el historial de transacciones, imposibilitando la auditoría posterior de operaciones pasadas o la generación de reportes comparativos anuales.
- Consistencia Referencial: Si un vehículo es eliminado físicamente, los análisis históricos que involucren modelos o marcas asociadas pueden arrojar discrepancias o errores de clave inexistente (*dangling references*).
- Recuperación ante Desastres / Ataques: El borrado accidental o malintencionado de datos es irreversible sin una copia de seguridad reciente. El borrado lógico permite la restauración inmediata del registro.

Implementación

El borrado lógico se implementó incorporando el atributo booleano "activo" en los documentos de la colección cars:

- Inclusión Inicial: Durante el proceso de migración en init_db.py, cada vehículo se inserta con el atributo "activo": True.
- Desactivación Lógica (crud_manager.py / mongo_bridge.py): Cuando un usuario solicita "borrar" un vehículo, el sistema ejecuta una operación de actualización que cambia el flag a False:

db.cars.update_one({_id: car_id}, {$set: {"activo": False}})

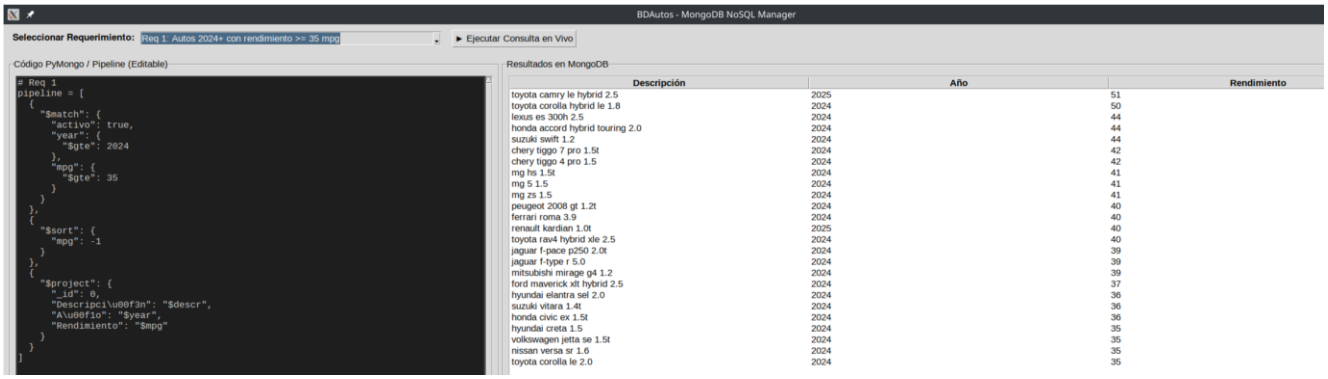
Filtrado Transparente en Consultas Analíticas: Todas las canalizaciones de agregación del catálogo (queries_catalog.py) inician obligatoriamente con una etapa de filtrado $match que excluye los documentos marcados como inactivos:

{ "$match": { "activo": true } }

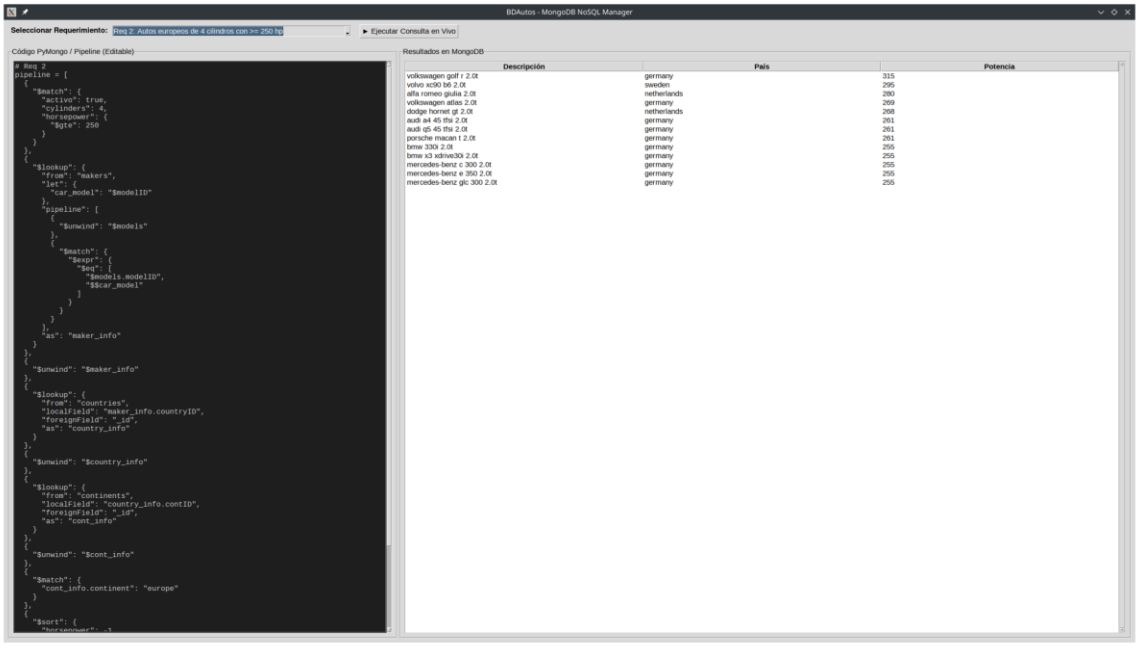
Esto garantiza que los vehículos deshabilitados no afecten los promedios, conteos ni métricas de los reportes gerenciales, pero permanezcan resguardados en la base de datos para su posterior restauración si es requerido.

Catálogo de Consultas y Evidencias

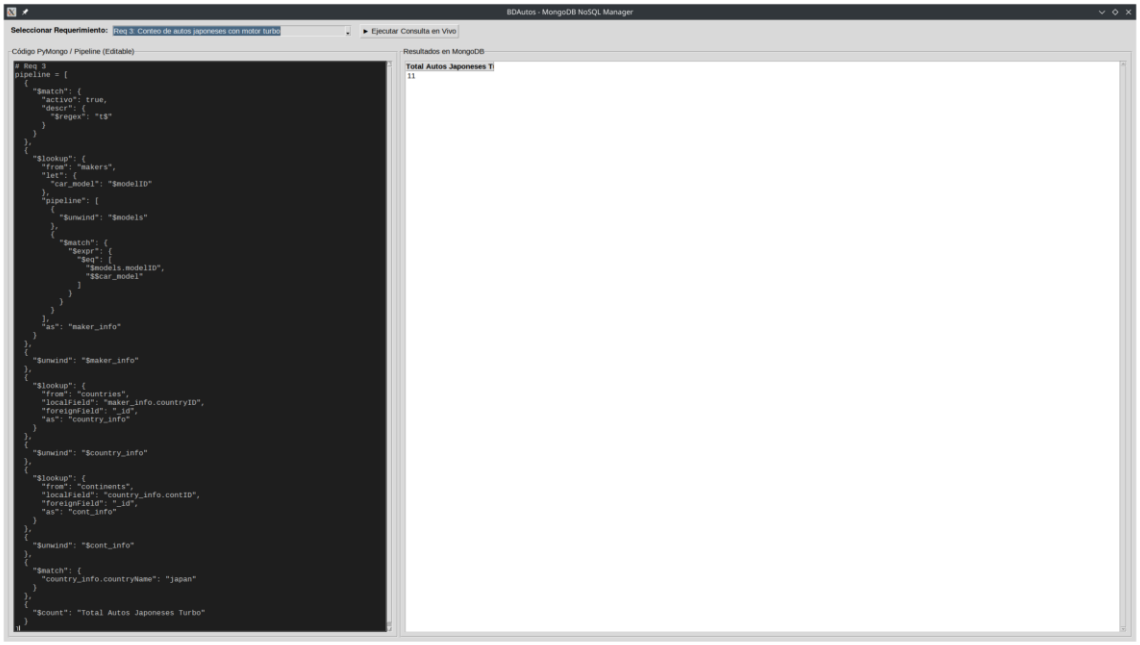
1. Obtener los autos de modelo 2024 en adelante con rendimiento de al menos 35 mpg. Mostrar solo la descripción, el año y el rendimiento, ordenados de mayor a menor rendimiento.



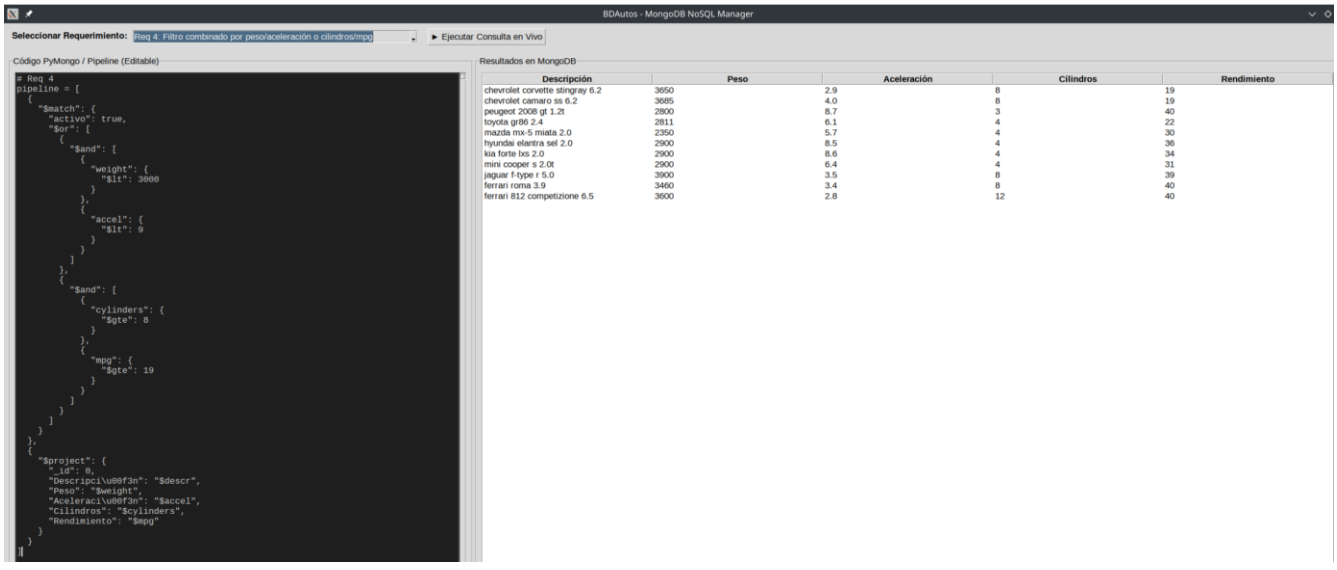
2. Listar los autos de fabricantes europeos con motor de 4 cilindros y al menos 250 hp, mostrando descripción, país y potencia, ordenados por potencia descendente.



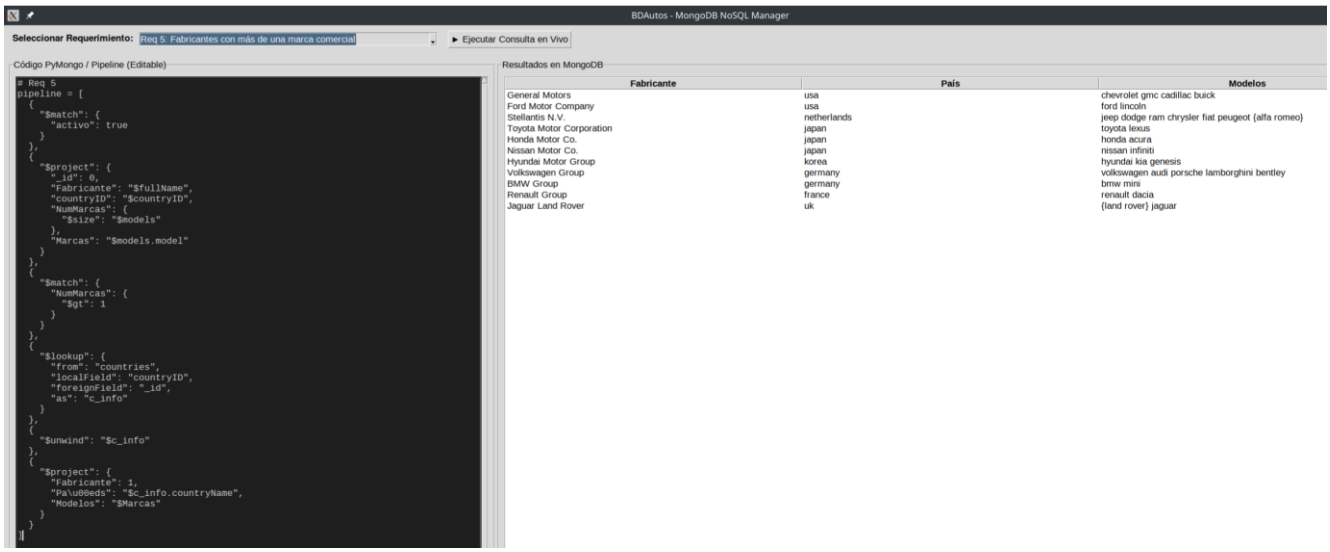
3. Contar cuántos autos de fabricantes japoneses tienen motor turbo, es decir, cuya descripción termina con la cilindrada seguida de "t" (por ejemplo "1.5t").



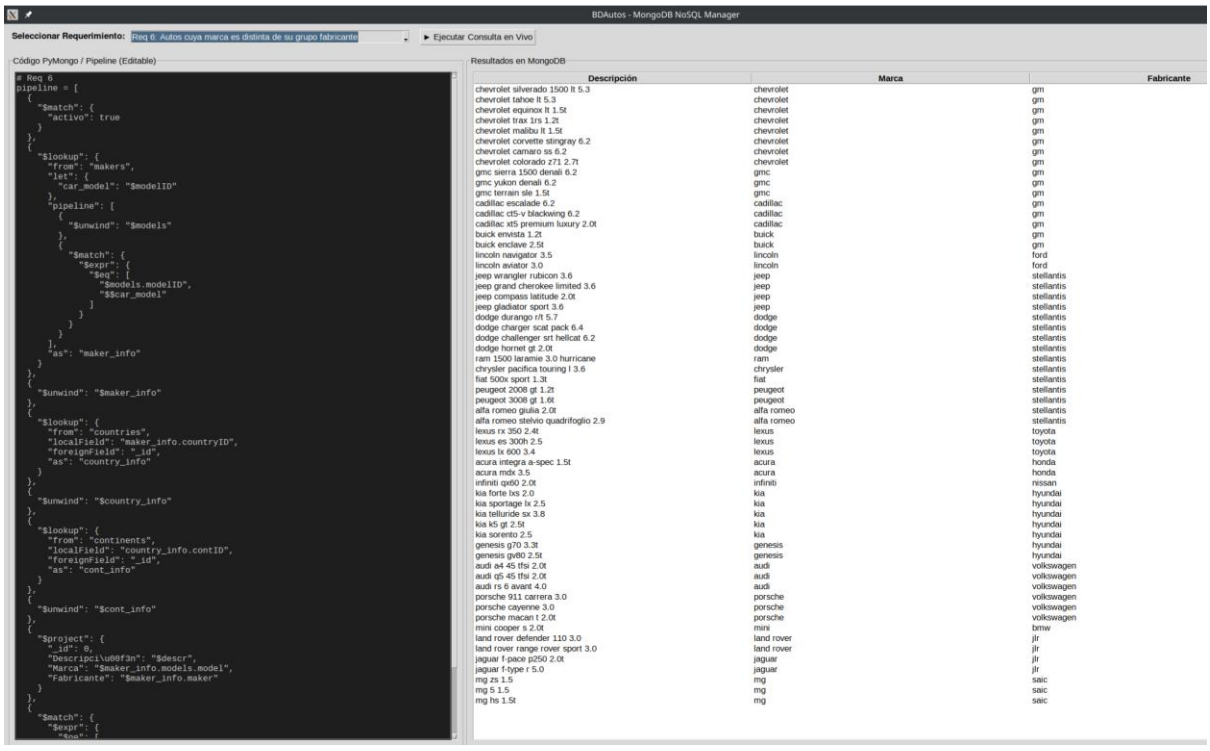
4. Encontrar los autos que cumplan cualquiera de estas condiciones: (a) pesan menos de 3000 lb y aceleran de 0 a 60 mph en menos de 9 s, o (b) tienen 8 o más cilindros y rinden al menos 19 mpg.



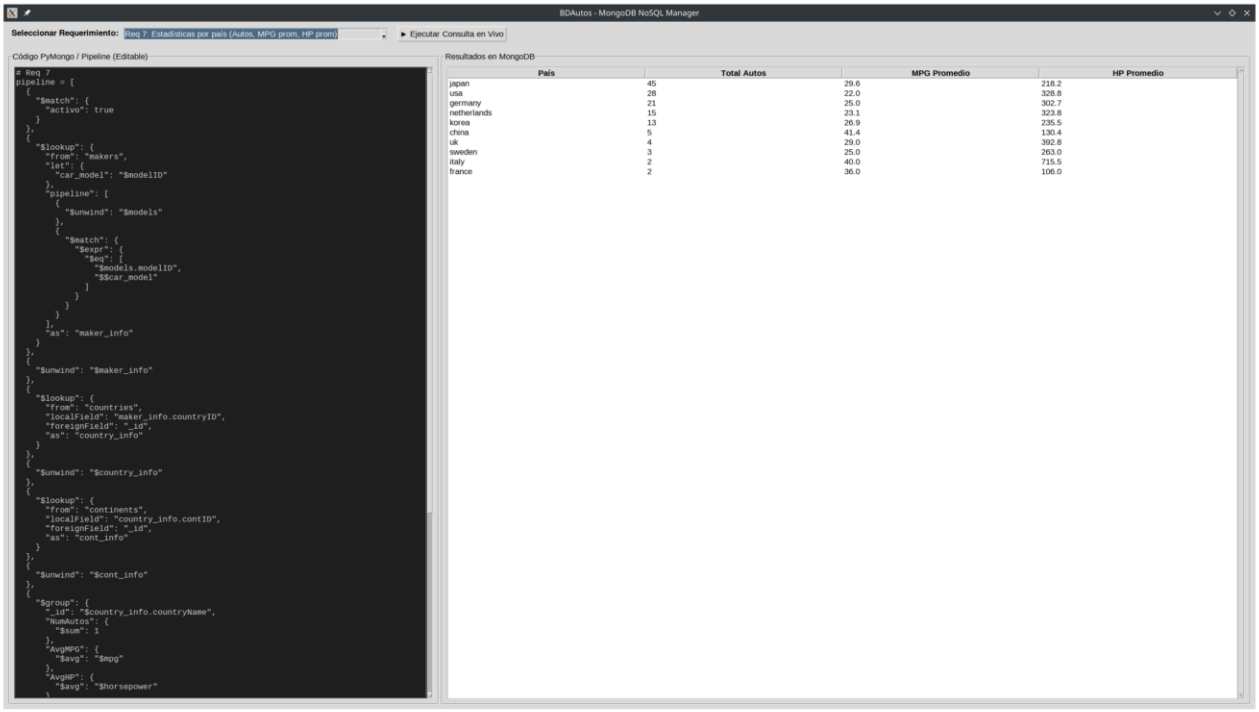
5. Obtener los fabricantes que comercializan más de una marca o línea de modelo, mostrando nombre, país y modelos.



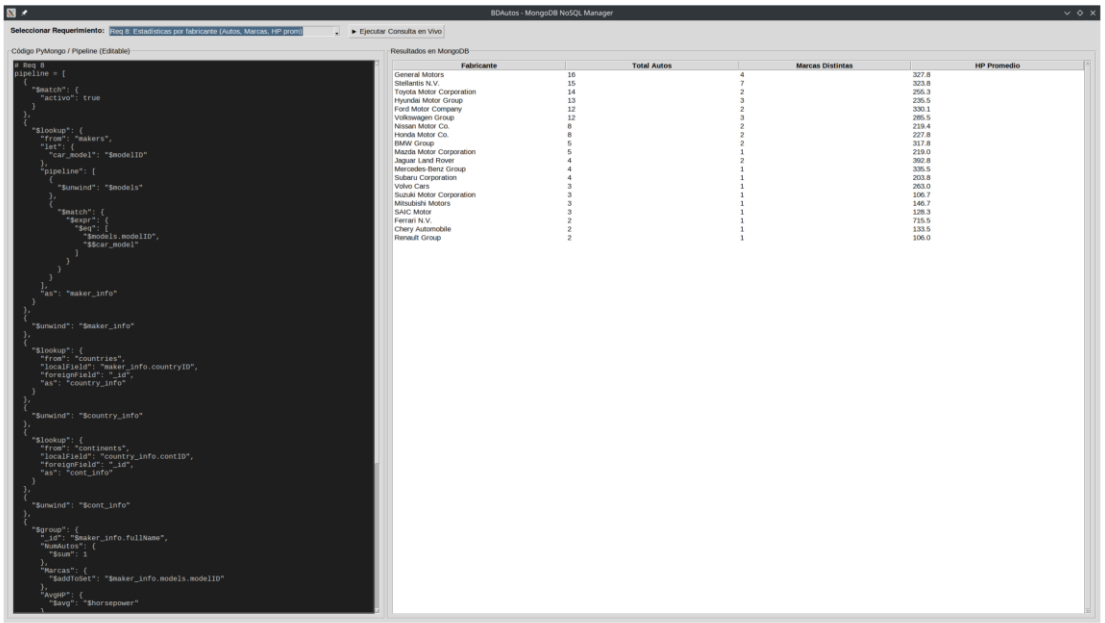
6. Encontrar los autos cuya marca (model.name) es distinta del nombre de su grupo fabricante (maker.name). Mostrar descripción, marca y fabricante.



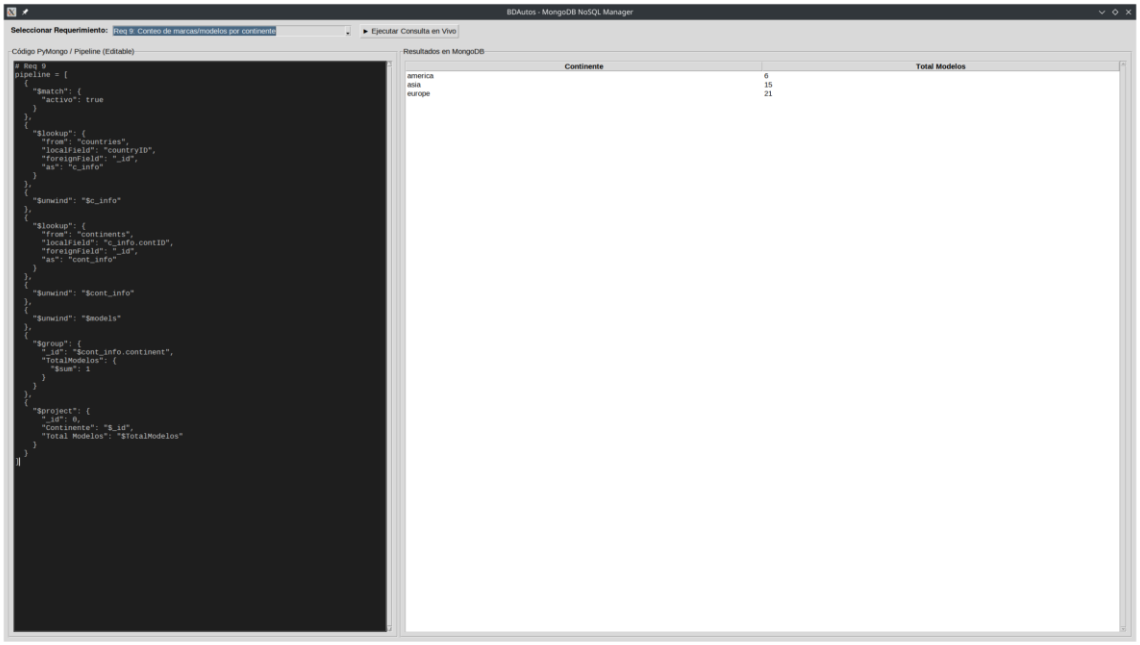
7. Calcular por país el número de autos, el rendimiento promedio y la potencia promedio (redondeados a 1 decimal), ordenado por número de autos.



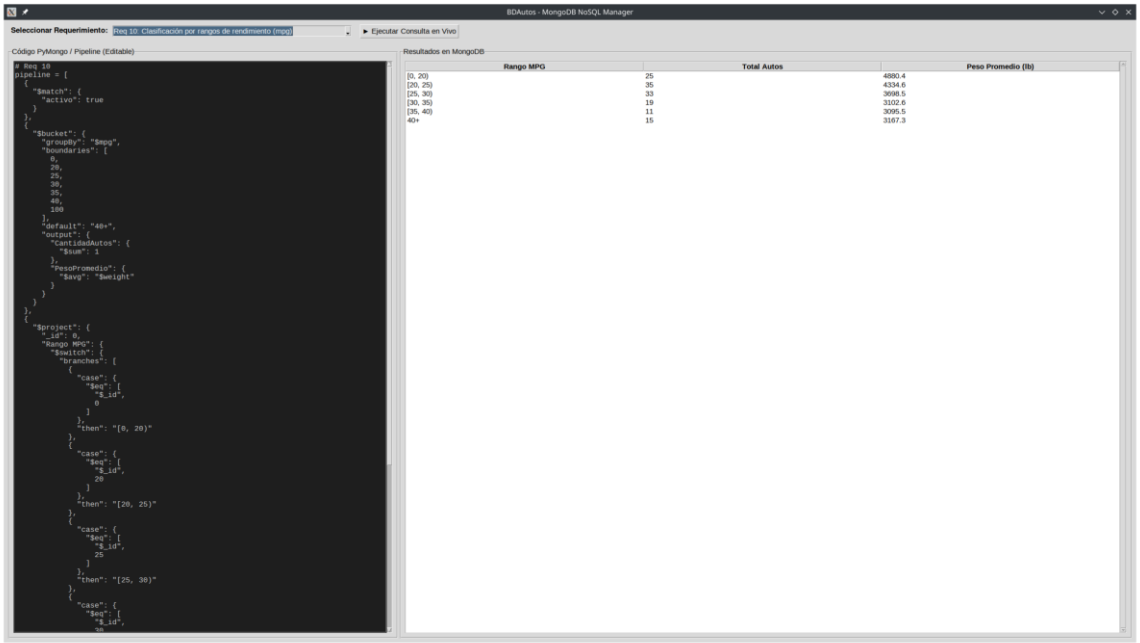
8. Calcular para cada grupo fabricante el número de autos, el número de marcas distintas con autos registrados y la potencia promedio (redondeada a 1 decimal), ordenado por número de autos.



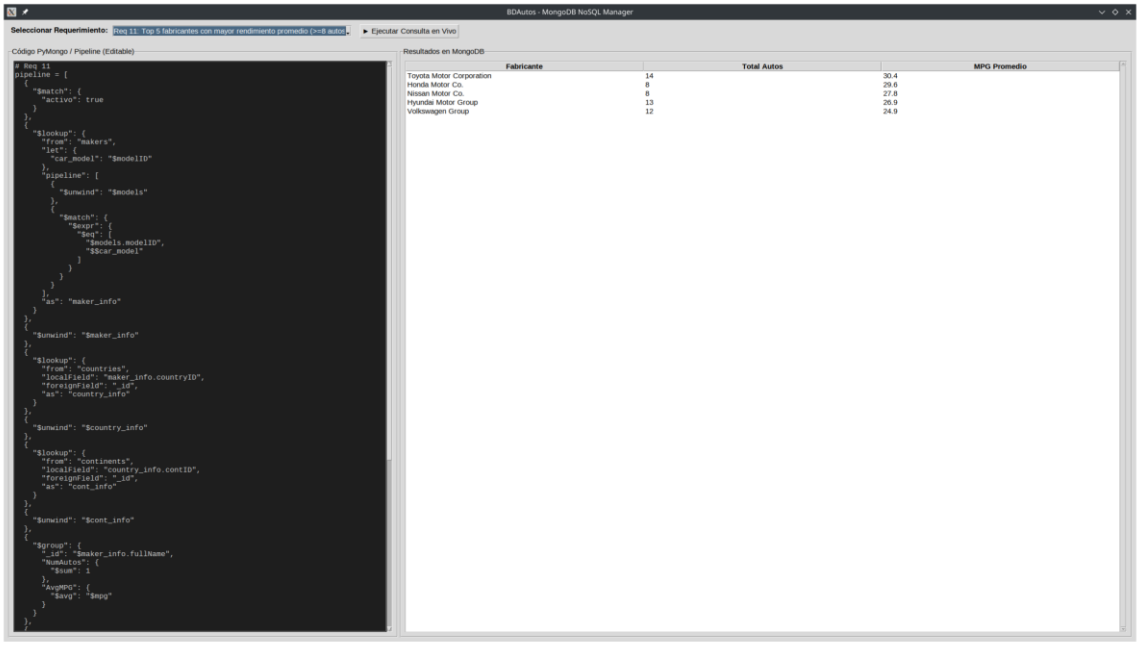
9. Contar cuántos modelos (marcas) hay por continente



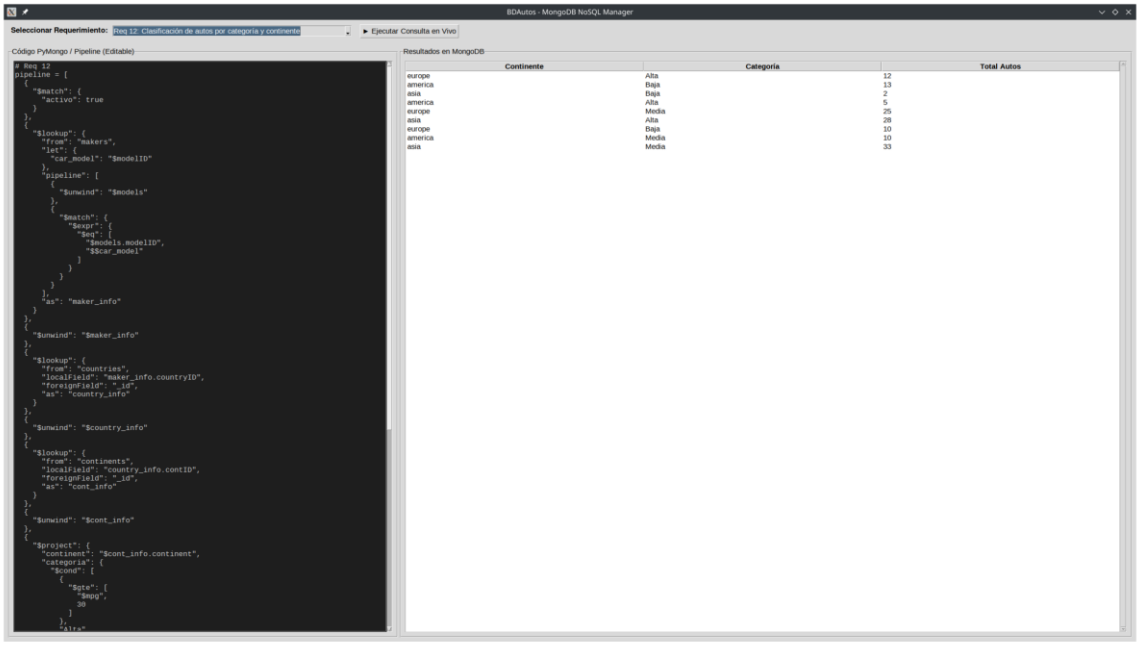
10. Clasificar los autos en rangos de rendimiento [0,20), [20,25), [25,30), [30,35), [35,40) y 40 o más, contando cuántos hay en cada rango y su peso promedio.



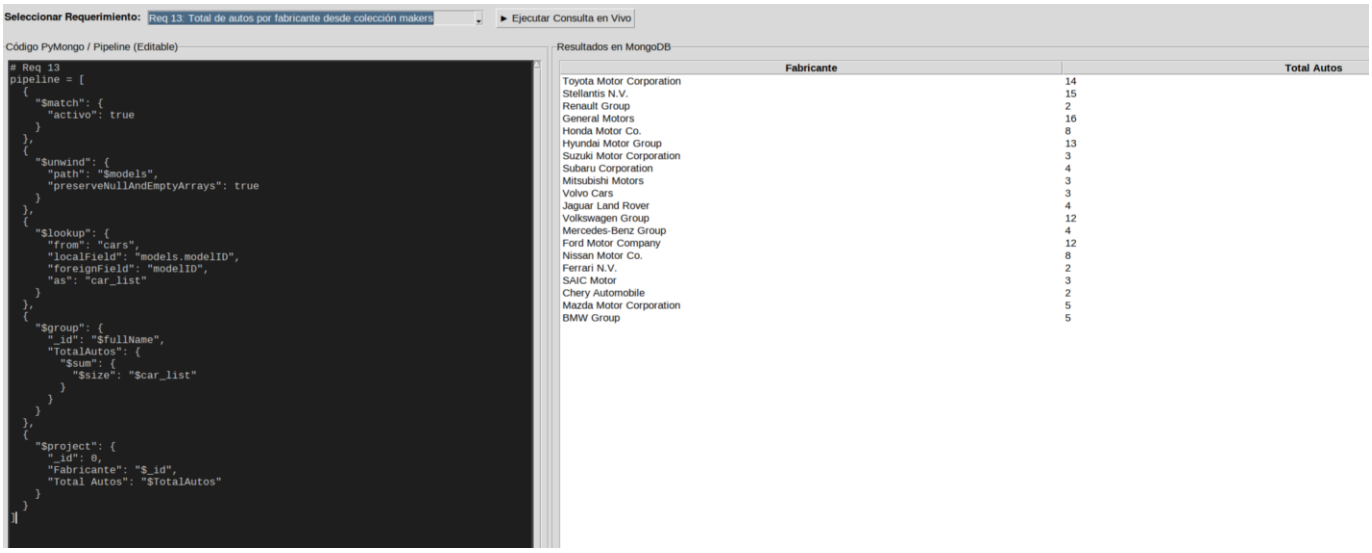
11. Obtener los 5 fabricantes con mayor rendimiento promedio, considerando solo aquellos con al menos 8 autos registrados.



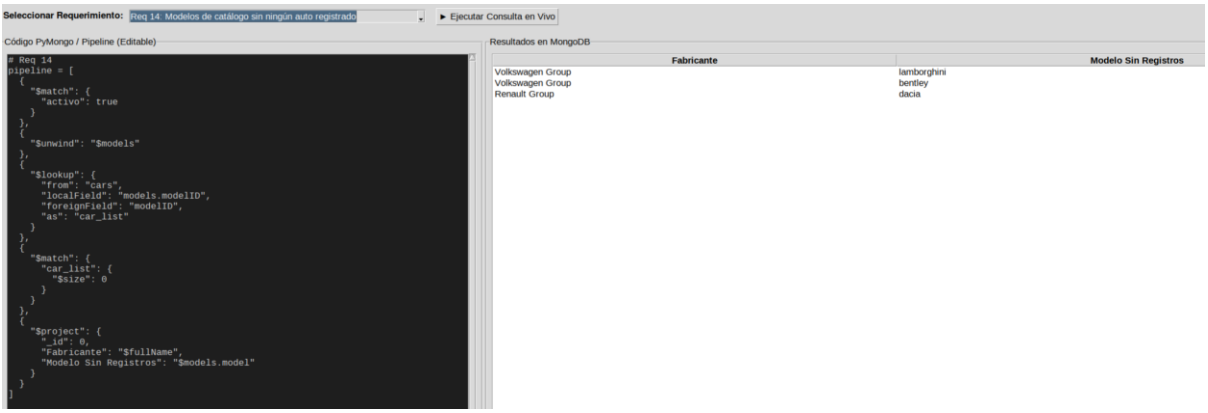
12. Clasificar cada auto como "Alta" (mpg ≥ 30), "Media" (20 ≤ mpg < 30) o "Baja" (mpg < 20) y contar cuántos autos hay por continente y categoría.



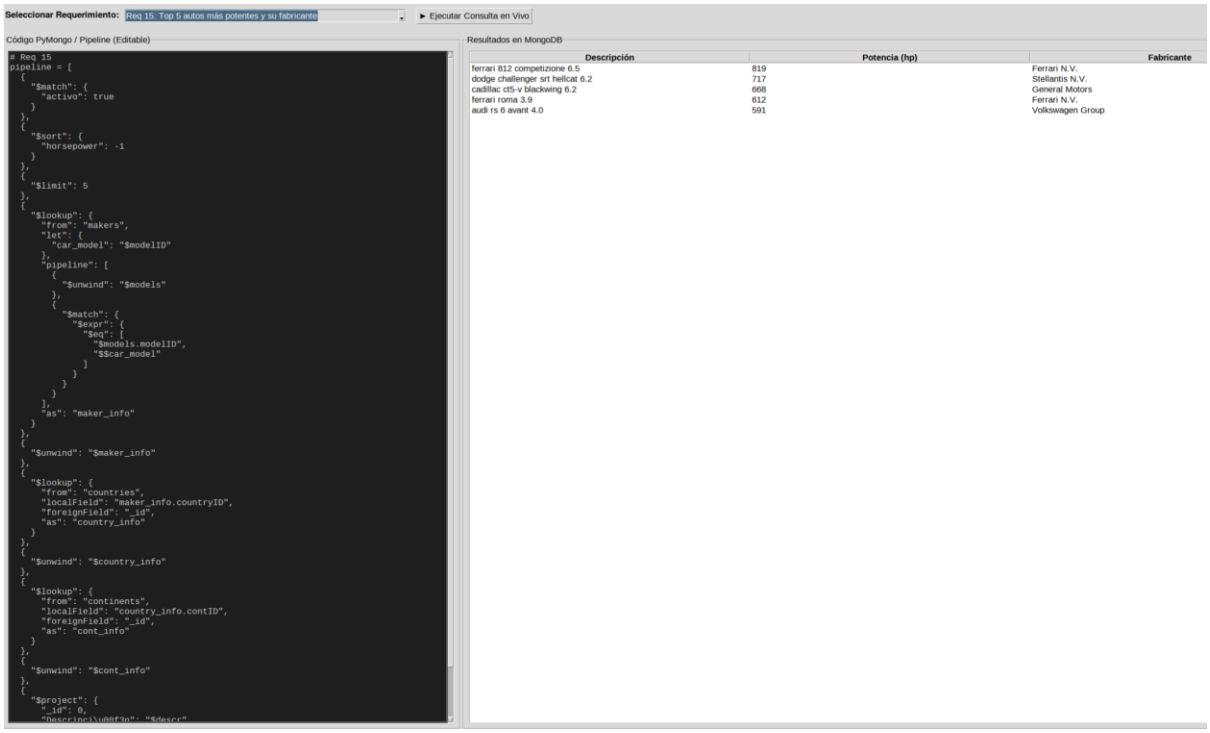
13. Desde la colección makers, obtener cuántos autos tiene registrados cada fabricante (incluidos los que tienen cero)



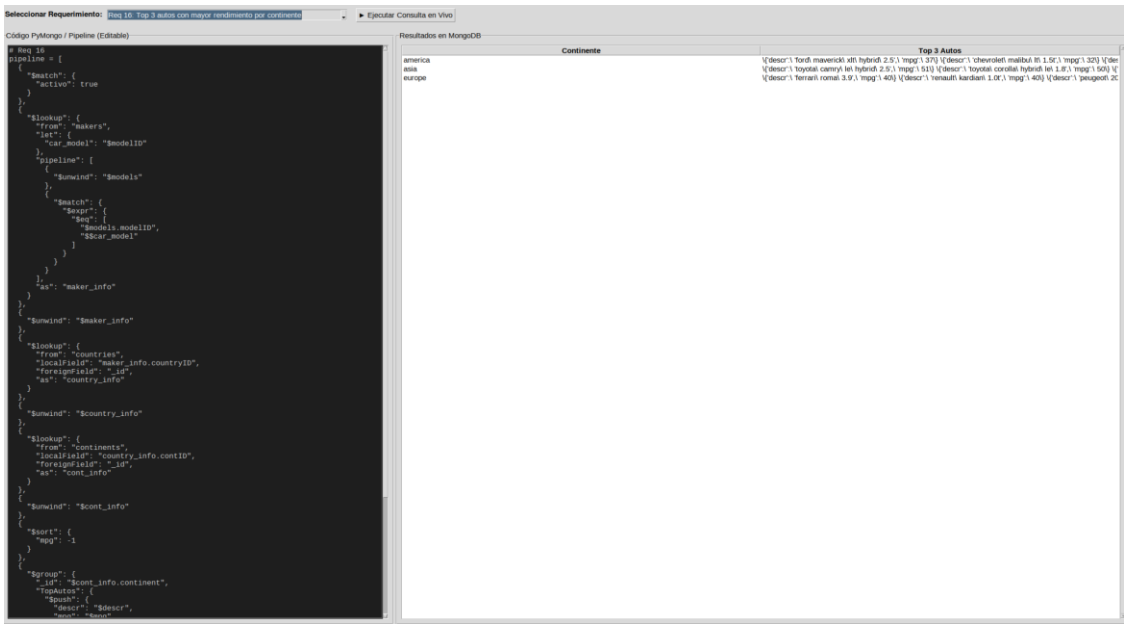
14. Encontrar los modelos del catálogo que no tienen ningún auto registrado



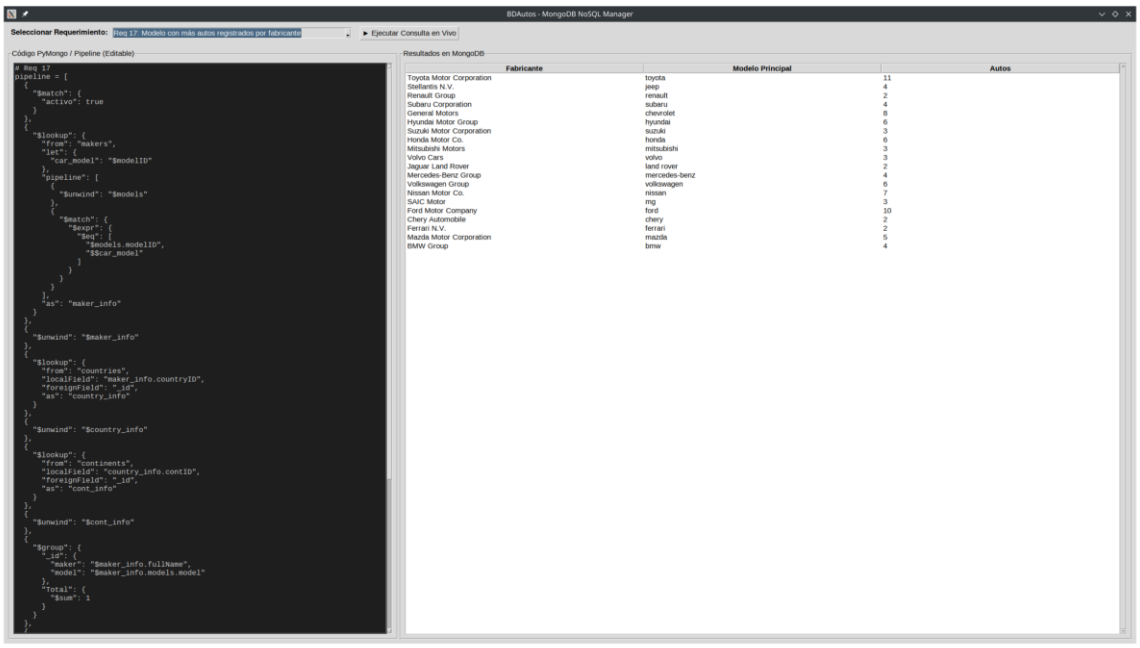
15. Para los 5 autos más potentes, obtener su descripción, potencia y el nombre completo (fullName) de su fabricante



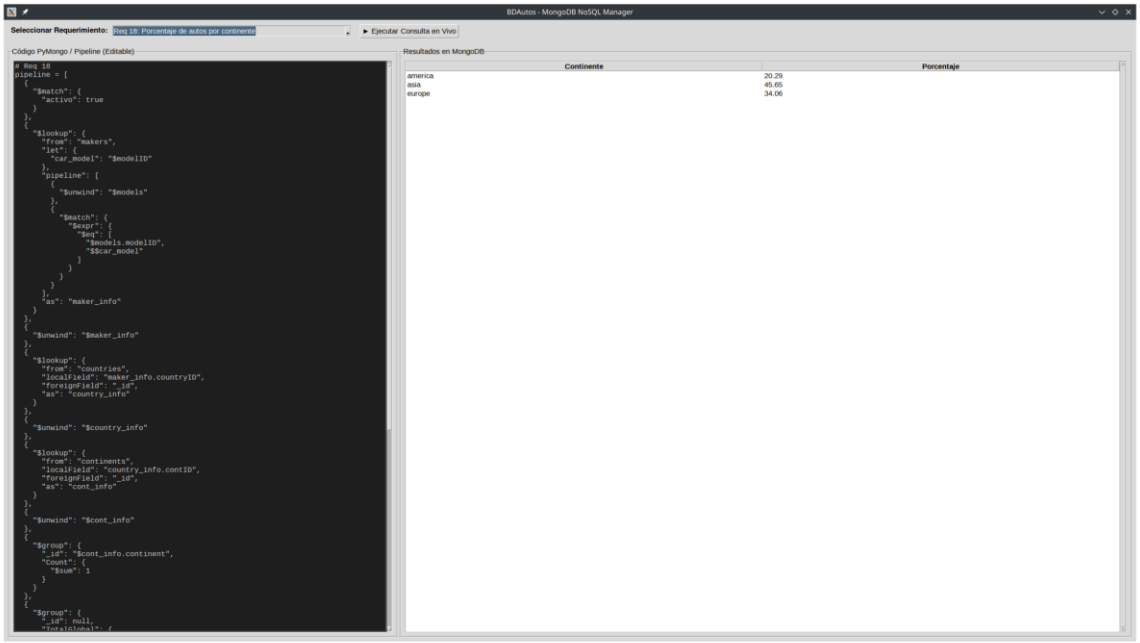
16. Obtener, para cada continente, los 3 autos con mayor rendimiento (descripción y mpg).



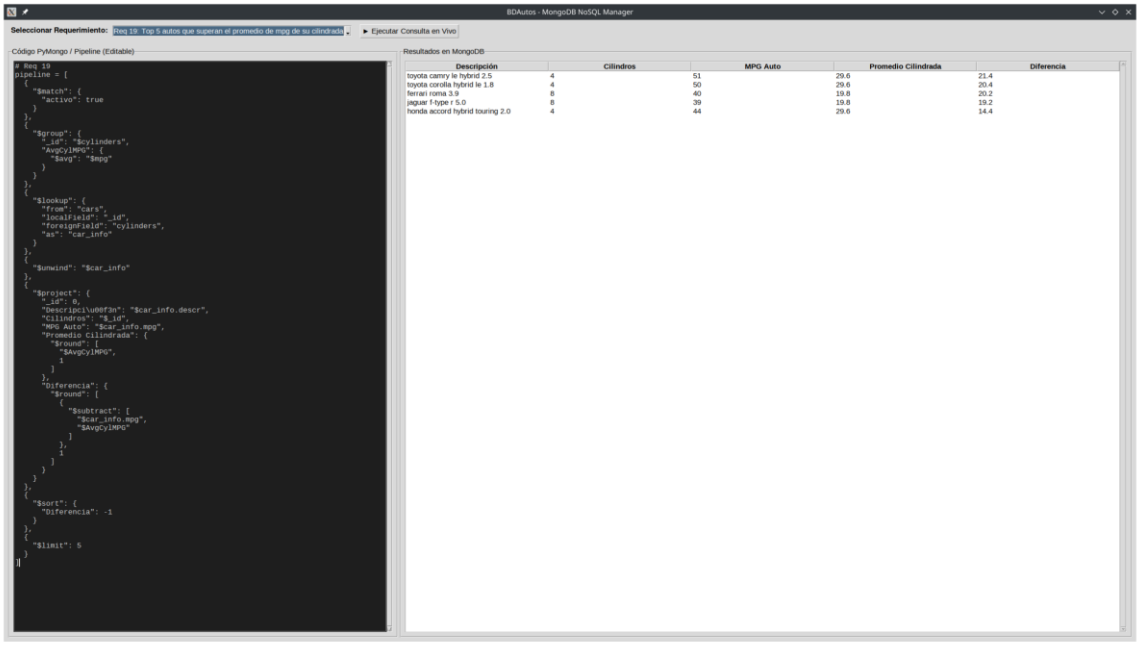
17. Para cada fabricante con más de un modelo, determinar cuál de sus modelos tiene más autos registrados.



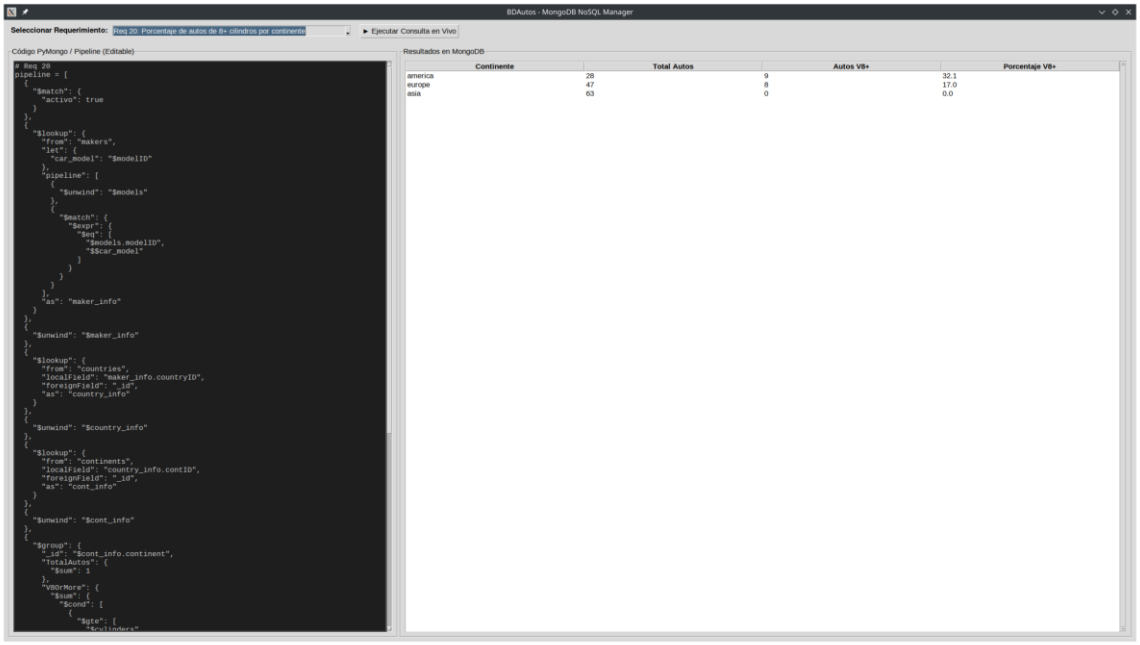
18. Calcular el porcentaje que representa cada continente sobre el total de autos, con 2 decimales.



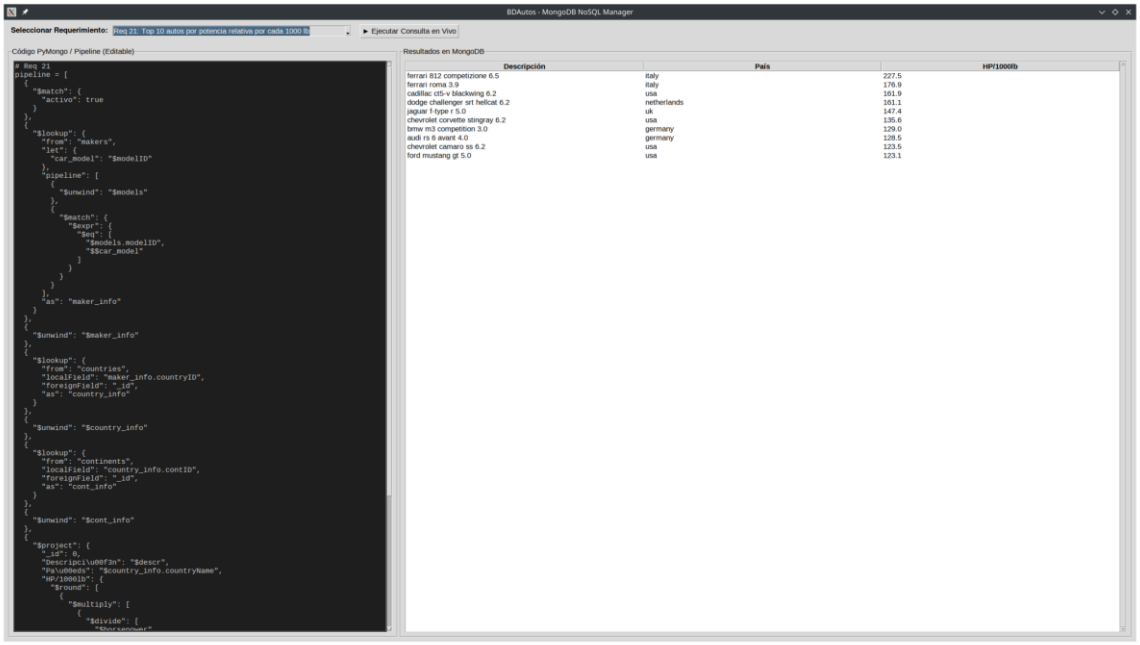
19. Listar los 5 autos que más superan el rendimiento promedio de los autos con su mismo número de cilindros, mostrando ese promedio y la diferencia.



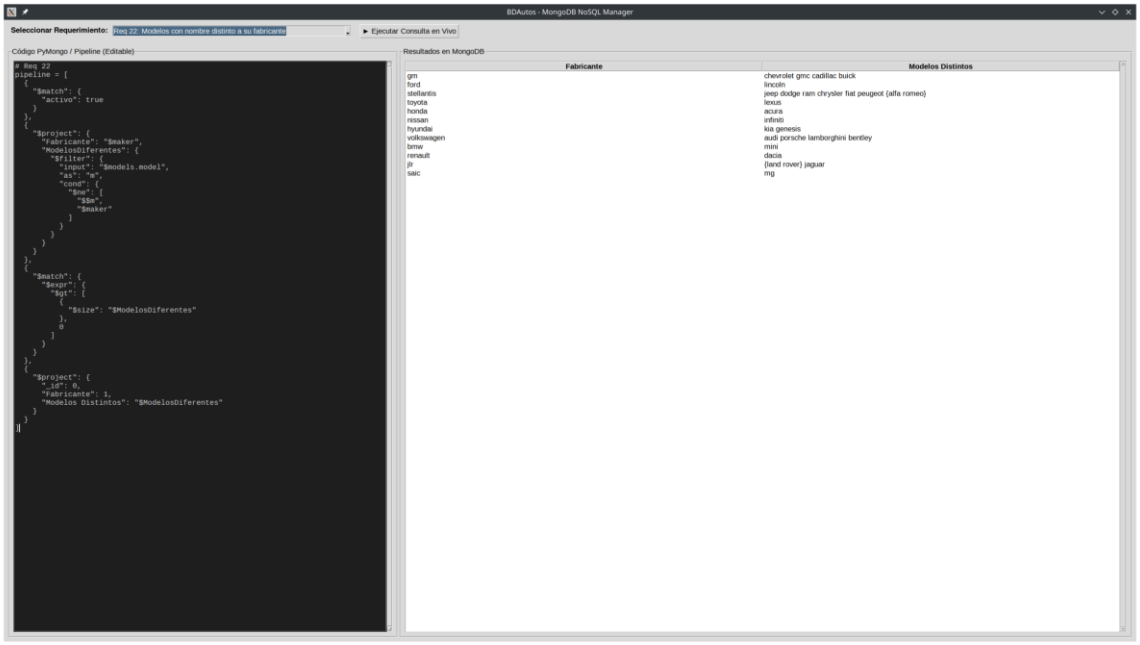
20. Calcular, por continente, el número de autos, cuántos tienen 8 o más cilindros y el porcentaje que representan (1 decimal).



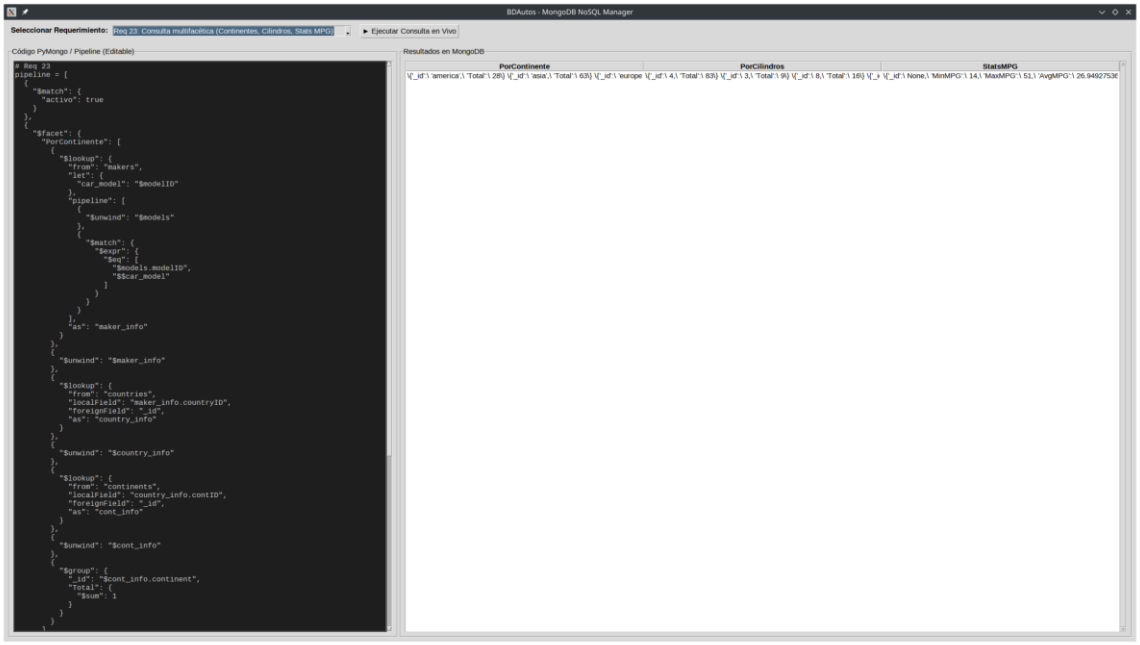
21. Calcular la potencia por cada 1000 lb de peso y mostrar los 10 autos con mayor valor, junto con su país.



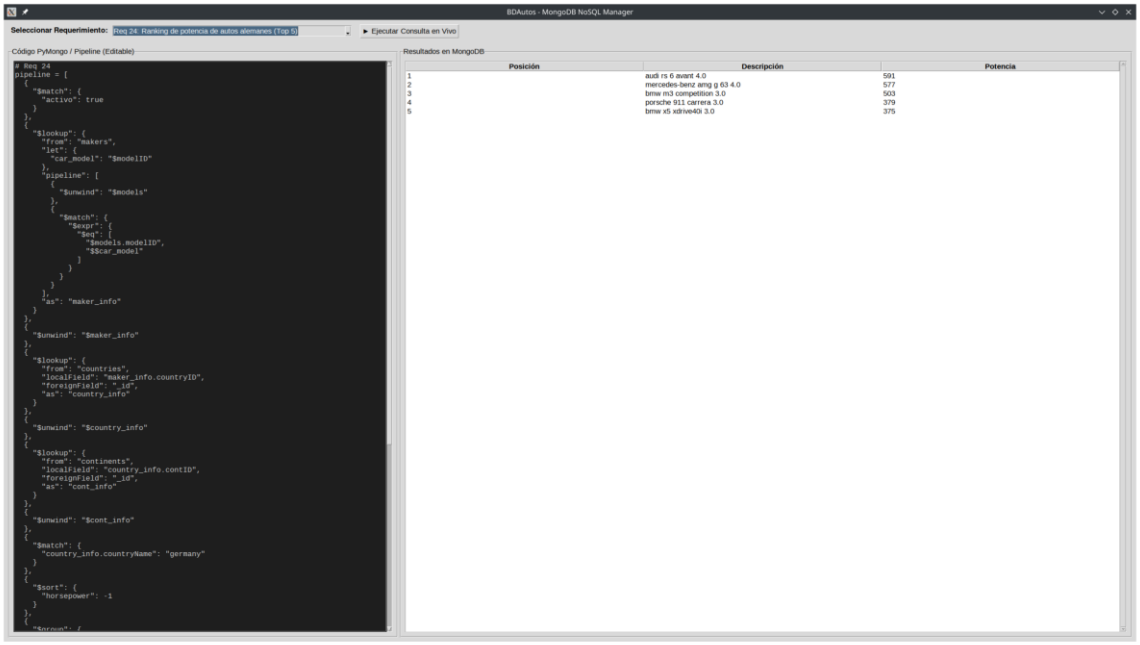
22. Para cada fabricante, mostrar solo los modelos cuyo nombre es distinto al nombre del fabricante, descartando a los fabricantes que no tengan ninguno.



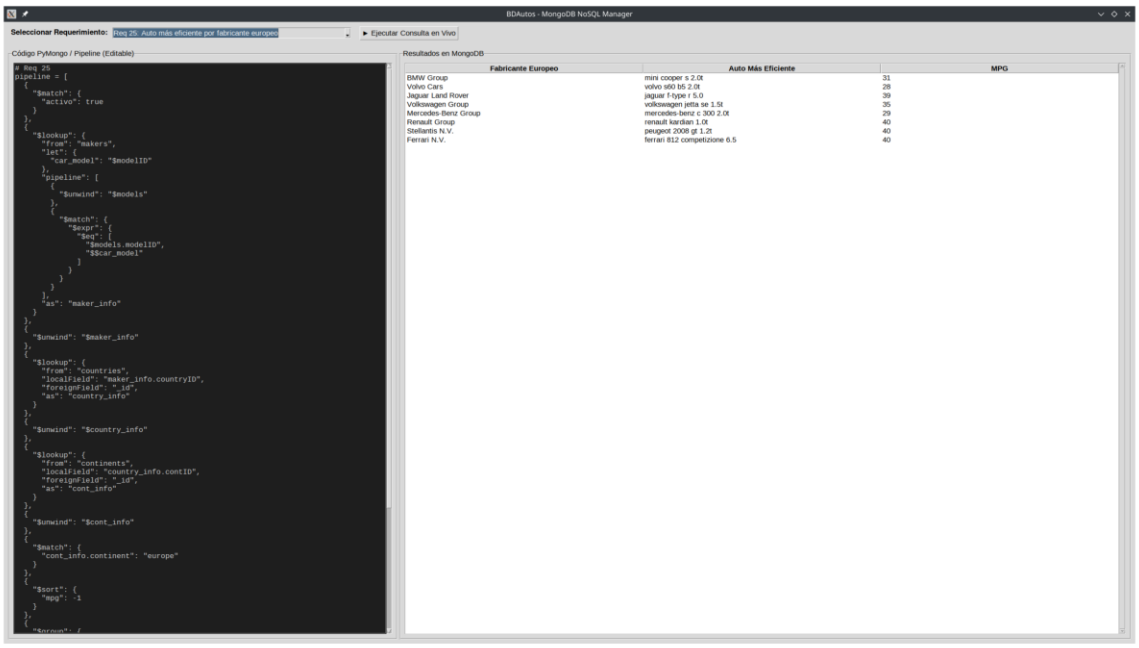
23. En una sola consulta, obtener: (a) el número de autos por continente, (b) el número de autos por cilindros y (c) las estadísticas globales de mpg (mínimo, máximo y promedio).



24. Asignar a cada auto alemán su posición por potencia dentro de su país y mostrar los 5 primeros.



25. Obtener para cada fabricante europeo su auto más eficiente



Conclusiones

El desarrollo del presente proyecto constituyó una validación práctica integral de los principios de modelamiento NoSQL y del diseño de sistemas analíticos sobre MongoDB mediante el lenguaje Python. La transición desde un paradigma relacional tradicional hacia un modelo orientado a documentos evidenció la eficacia de la desnormalización estratégica. Al consolidar descripciones y especificaciones técnicas dentro de la colección cars mediante el embebido de datos, se eliminó la dependencia de operaciones de unión (*joins*) computacionalmente costosas, logrando que la información crítica para el análisis resida de forma contigua en documentos autosuficientes y optimizando los tiempos de respuesta del sistema.

Asimismo, la exploración sistemática del *Aggregation Framework* —mediante la articulación de etapas como $match, $group, $project, $sort y $lookup— demostró la capacidad del motor de base de datos para ejecutar procesamientos analíticos complejos de forma nativa. Esta delegación de cómputo hacia el servidor garantizó que la aplicación cliente en Python recibiera únicamente conjuntos de datos consolidados, reduciendo drásticamente la huella de memoria y la sobrecarga en la red. Dicha eficiencia se potenció mediante una arquitectura modular basada en el patrón *Code as Object*, la cual facilitó la integración fluida entre el controlador PyMongo y la interfaz gráfica desarrollada en Tkinter. Al desacoplar el catálogo de consultas analíticas en un módulo independiente, la aplicación asumió un comportamiento reactivo equivalente al de herramientas de inspección avanzadas como MongoDB Compass, permitiendo la previsualización y ejecución de canalizaciones BSON en tiempo real.

Finalmente, la incorporación del patrón de borrado lógico en el módulo de gestión crud_manager.py acentuó la importancia de priorizar la integridad y la trazabilidad de la información en entornos de producción reales. Esta estrategia demostró cómo es posible salvaguardar la seguridad y la reversibilidad de los registros ante eventuales auditorías o fallos, sin comprometer la precisión de los reportes analíticos generados por las canalizaciones de agregación. En conjunto, la solución implementada no solo ratificó la viabilidad de MongoDB como infraestructura sólida para la gestión analítica y operativa, sino que también consolidó un marco de desarrollo riguroso, escalable y mantenible.

Repositorio

https://github.com/a364693-cmd/Proyecto_2Parcial-BDAutos-MongoDB/blob/main/README.md

Bibliografía

- **MongoDB, Inc.** (s. f.). *MongoDB documentation: Aggregation operations*. MongoDB Manual. https://www.mongodb.com/docs/manual/aggregation/

- **PyMongo Developers.** (s. f.). *PyMongo documentation: Python driver for MongoDB*. Read the Docs. https://pymongo.readthedocs.io/

- **Code Lucky.** (s. f.). *Buscar: Mongo* [Canal de YouTube]. YouTube. Recuperado el 1 de noviembre de 2026, de https://www.youtube.com/@thecodelucky/search?query=mongo

- **Code Lucky.** (s. f.). *Code Lucky: Tutoriales y desarrollo de software*. https://codelucky.com/

- **CodeWolfy.** (s. f.). *Mastering MongoDB with Python and PyMongo*. CodeWolfy. https://codewolfy.com/mastering-mongodb-with-python-and-pymongo/

- **GeeksforGeeks.** (2023, 16 de mayo). *Aggregation pipeline stages in MongoDB*. GeeksforGeeks. https://www.geeksforgeeks.org/mongodb/aggregation-pipeline-stages-in-mongodb-set-1/

- **Tutoriales de Programación.** (s. f.). *MongoDB and Python tutorial for beginners: Getting started with PyMongo* [Video]. YouTube. https://www.youtube.com/watch?v=XAQQL0-uToc